



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 03

NOMBRE COMPLETO: Bueno Hernández Héctor Daniel

N° de Cuenta: 319233148

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 04

SEMESTRE 2025-2

FECHA DE ENTREGA LÍMITE: 01/03/2025

CALIFICACIÓN: _____

Construcción de la pirámide

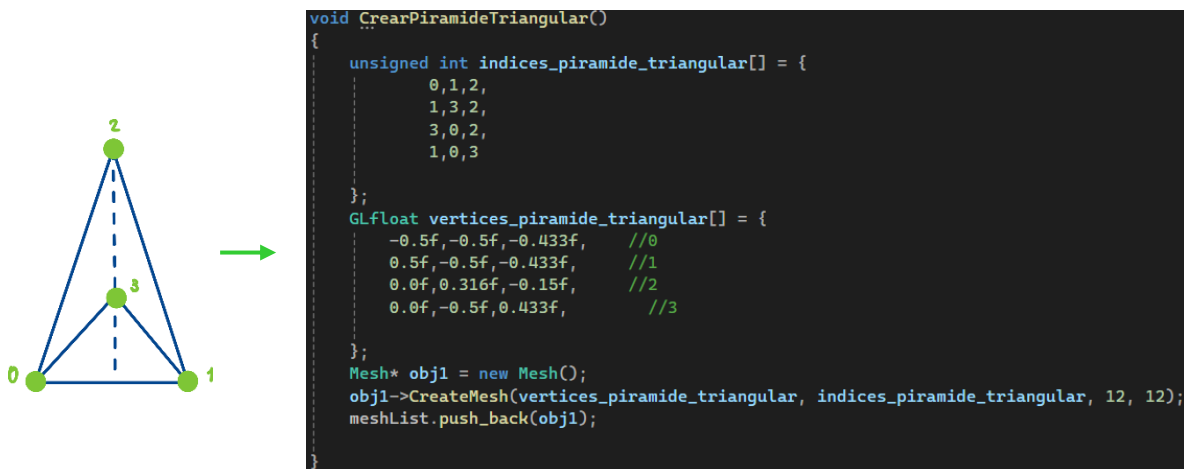
Se solicitó construir una Pirámide Rubik, también conocida como *Pyraminx*, haciendo uso de pirámides. Para ello, lo primero que hice fue modificar los vértices de la pirámide contenidos en el código proporcionado por el profesor debido a que yo buscaba que cada arista de la pirámide midiera una unidad.

A este tipo especial de pirámides se les llama *tetraedro triangular* y sus medidas de ancho y alto son las siguientes:

$$\text{Alto} = \frac{\sqrt{2}}{\sqrt{3}} \approx 0.816$$

$$\text{Ancho} = \frac{\sqrt{3}}{2} \approx 0.866$$

Por lo tanto, tomando en cuenta estos valores y el siguiente mapeo para los vértices, los valores de estos se modificaron de la siguiente forma:



Una vez que los vértices fueron establecidos adecuadamente, primero se construyó una pirámide negra grande que actuaría como la base para incrustar en ella las pirámides correspondientes a los stickers de colores del Pyraminx. Cabe destacar que se utilizó la traslación opcional en todas las pirámides, la cual fue proporcionada ya en el código y esta sirvió para establecer el origen antes de realizar las rotaciones correspondientes:

El código correspondiente a la pirámide negra es el siguiente:

```
model = glm::mat4(1.0f);
color = glm::vec3(0.0f, 0.0f, 0.0f);
//Opcional duplicar esta traslación inicial para posicionar en -Z a los objetos en el mismo punto
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.0f, 0.34f, 0.287f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[1]->RenderMesh();
```

Cara amarilla

Posterior a esto, primero se hizo la construcción de la cara inferior de la pirámide en color amarillo debido a que esta era la que presentaba el menor nivel de dificultad. Para cada pirámide pequeña se escalaron los ejes 0.4 veces su tamaño original y se incrustaron a la pirámide negra haciendo uso de matrices `translate`.

Los bloques de código correspondientes a las pirámides amarillas son los siguientes:

```
//PIRÁMIDES AMARILLAS

model = glm::mat4(1.0f);
color = glm::vec3(1.0f, 1.0f, 0.0f);
//Opcional duplicar esta traslación inicial para posicionar en -Z a los objetos en el mismo punto
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.0f, -0.05f, -0.015f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[1]->RenderMesh();

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.5f, -0.05f, -0.015f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(-0.5f, -0.05f, -0.015f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(-0.25f, -0.05f, 0.415f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.25f, -0.05f, 0.415f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();
```

```

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.0f, -0.05f, 0.845f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

```

Para las pirámides restantes fue necesario aplicar una rotación adicional de 180° en el eje Y para que todas las pirámides encajaran adecuadamente:

```

////PIRÁMIDES AMARILLAS VOLTEADAS

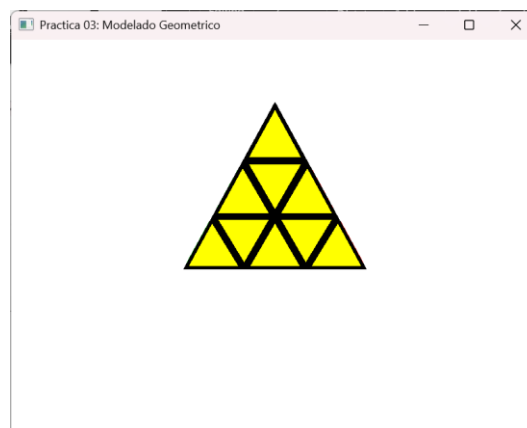
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::translate(model, glm::vec3(0.25f, -0.05f, -0.015f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::translate(model, glm::vec3(-0.25f, -0.05f, -0.015f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::translate(model, glm::vec3(0.0f, -0.05f, -0.445f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

```

Ya en ejecución, esta cara se ve de la siguiente forma:



Cara azul

Después se hizo la cara azul de la pirámide ya que en esta se encontraba el origen de mi figura, por lo que no fue difícil el trasladar las pirámides más pequeñas a su posición adecuada.

```
//PIRÁMIDES AZULES

model = glm::mat4(1.0f);
color = glm::vec3(0.0f, 0.0f, 0.9f);
//Opcional duplicar esta traslación inicial para posicionar en -Z a los objetos en el mismo punto
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.0f, -0.01f, -0.05f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[1]->RenderMesh();

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.5f, -0.01f, -0.05f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(-0.5f, -0.01f, -0.05f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(-0.25f, 0.4f, 0.09f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.25f, 0.4f, 0.09f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.0f, 0.81f, 0.235f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();
```

Para poder hacer que las pirámides rotaran a su orientación correspondiente, fue necesario agregar dos rotaciones adicionales a las ya establecidas anteriormente: una rotación de 180° en el eje Z para que la pirámide estuviera de cabeza y una rotación de -42° en el eje X para que una de las caras tuviese la misma inclinación que las caras de la pirámide negra.

Para que las pirámides giraran sobre su propio eje, la traslación que pone a la pirámide en su posición adecuada se colocó antes de realizar las últimas rotaciones ya que, de lo contrario, la pirámide cambiaba su posición al mismo tiempo que su orientación cuando se realizaba la rotación.

Los bloques de código generados fueron los siguientes:

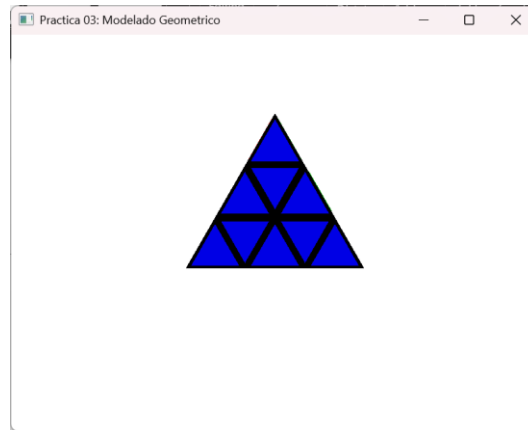
```
//PIRÁMIDES AZULES ROTADAS

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.25f, -0.13f, -0.1f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, -42 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(-0.25f, -0.13f, -0.1f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, -42 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.0f, 0.29f, 0.05f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, -42 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();
```

Ya en ejecución, esta cara se ve de la siguiente forma:



Cara roja

Para las pirámides que no se encuentran rotadas de esta cara, el código es básicamente el mismo que el usado en la cara azul, solo cambian las coordenadas a las que fueron trasladadas las pirámides para que se encontraran en su posición adecuada.

```
//PIRÁMIDES ROJAS

model = glm::mat4(1.0f);
color = glm::vec3(1.0f, 0.0f, 0.0f);
//Opcional duplicar esta traslación inicial para posicionar en -Z a los objetos en el mismo punto
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.28f, -0.01f, 0.43f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[1]->RenderMesh();

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.53f, -0.01f, 0.0f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.03f, -0.01f, 0.86f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();
```

```

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.28f, 0.4f, 0.135f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.03f, 0.4f, 0.575f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.03f, 0.81f, 0.28f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

```

Por otro lado, para las pirámides que se encuentran rotadas fue necesario agregar una rotación en el eje Y adicional a las rotaciones ya establecidas en la cara azul. Esto con el propósito de rotar la cara que cuenta con la misma inclinación que la pirámide negra.

Para poder lograrlo, primero me basé en la rotación aplicada a las pirámides amarillas y posteriormente apliqué las mismas rotaciones antes mencionadas sobre los ejes restantes. En este caso, fue necesario trasladar la pirámide al centro, rotarla en Y y volverla a trasladar antes de aplicar las últimas rotaciones. Si realizaba las rotaciones en los ejes X y Z antes de trasladarla por segunda vez, los ejes de la pirámide rotan al igual que la figura, por lo que resultaría muy complicado poder trasladarla a la posición deseada.

Finalmente, los bloques de código generados fueron los siguientes:

```

//PIRAMIDES ROJAS ROTADAS

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, -120 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f)); //--
model = glm::translate(model, glm::vec3(0.175f, 0.29f, -0.28f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f)); //--
model = glm::rotate(model, -42 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f)); //--
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

```



```

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, -120 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f)); //--
model = glm::translate(model, glm::vec3(-0.07f, -0.13f, -0.42f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f)); //--
model = glm::rotate(model, -42 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f)); //--
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

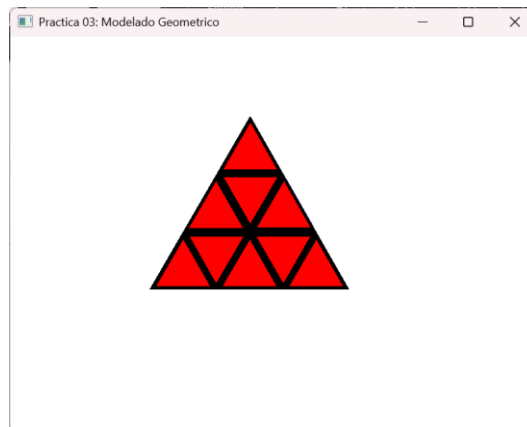
```

```

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, -120 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f)); //--
model = glm::translate(model, glm::vec3(0.43f, -0.13f, -0.42f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f)); //--
model = glm::rotate(model, -42 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f)); //--
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

```

Ya en ejecución, esta cara se ve de la siguiente forma:



Cara verde

Para la construcción de esta cara se usó el mismo procedimiento que para la construcción de la cara roja. Las pirámides que no están rotadas simplemente fueron trasladadas a la posición deseada y a las pirámides rotadas se les aplicaron las mismas transformaciones que a las pirámides rojas.

Una vez que se completó la cara roja, el realizar esta cara fue muy sencillo ya que solo fue necesario invertir los valores de la rotación en Y junto con las traslaciones para que estas se colocaran adecuadamente.

Los bloques de código generados fueron los siguientes:

```
//PIRÁMIDES VERDES
```

```
model = glm::mat4(1.0f);
color = glm::vec3(0.0f, 0.7f, 0.0f);
//Opcional duplicar esta traslación inicial para posicionar en -Z a los objetos en el mismo punto
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(-0.28f, -0.01f, 0.43f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[1]->RenderMesh();
```

```
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(-0.53f, -0.01f, 0.0f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();
```

```
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(-0.03f, -0.01f, 0.86f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();
```

```
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(-0.285f, 0.4f, 0.145f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();
```

```
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(-0.03f, 0.4f, 0.575f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();
```

```
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(-0.03f, 0.81f, 0.285f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();
```

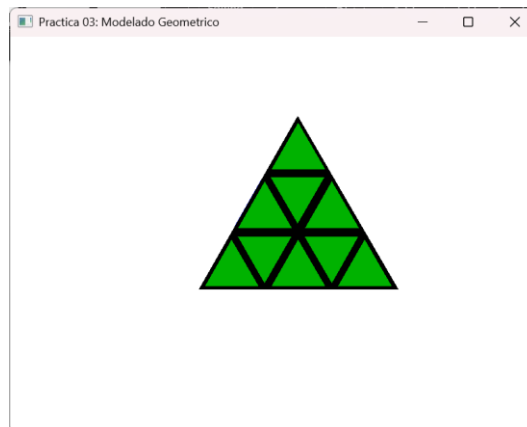
```
//PIRÁMIDES VERDES ROTADAS

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, 120 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f)); //--
model = glm::translate(model, glm::vec3(-0.175f, 0.29f, -0.28f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f)); //--
model = glm::rotate(model, -42 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f)); //--
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));//FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

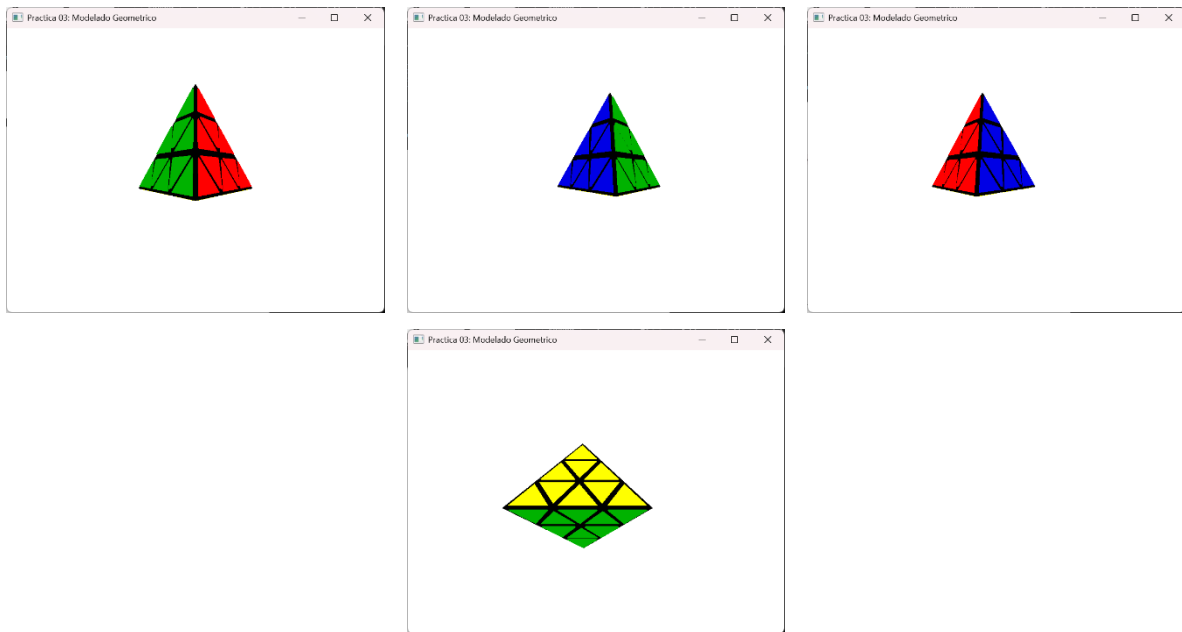
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, 120 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f)); //--
model = glm::translate(model, glm::vec3(0.07f, -0.13f, -0.42f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f)); //--
model = glm::rotate(model, -42 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f)); //--
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));//FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, 120 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f)); //--
model = glm::translate(model, glm::vec3(-0.43f, -0.13f, -0.42f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f)); //--
model = glm::rotate(model, -42 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f)); //--
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));//FALSE ES PARA QUE NO SEA TRANSPUESTA
meshList[1]->RenderMesh();
```

Ya en ejecución, esta cara se ve de la siguiente forma:



A continuación, también se muestran más ángulos de la pirámide donde se puede apreciar más de una cara a la vez:



Problemas presentados

A lo largo de la realización de la práctica no tuve problemas de compilación en ningún momento, sin embargo, tuve muchos problemas al momento de rotar las pirámides de colores.

Si bien hacer la cara amarilla fue sencillo, en las demás caras me resultó muy complicado encontrar la forma adecuada de realizar las rotaciones sin que la pirámide en cuestión se moviera de forma exagerada. Asimismo, también tuve algunos problemas con las traslaciones ya que, después de rotar la pirámide, los ejes también se movían, provocando así que el calcular los valores de las traslaciones me fuese mucho más difícil.

Para poder resolver este problema estuve jugando con el orden de las transformaciones, pero no fue hasta que realmente analicé el sentido de las rotaciones y la relación entre estas y traslaciones que me fue más fácil encontrar la forma de girarlas sin que se movieran. Además, esto me ayudó a darme cuenta de que cada cara iba a necesitar una combinación distinta de transformaciones y que no era simplemente trasladarlas.

Fuera de esto, la única otra complicación que llegué a presentar fue que había ocasiones en las que me costaba acomodar las pirámides en sus posiciones debido a que el centro parecía estar desfasado, por lo que tuve que estarlas moviendo al tanteo hasta que quedaran en la posición que yo quería y esto me consumió mucho tiempo.

Conclusiones

La complejidad de la práctica en lo personal me pareció un poco elevada ya que en clase no habíamos trabajado previamente con rotaciones en los tres ejes de forma simultánea. Si bien el profesor nos explicó la relación entre la rotación y traslación para que las figuras giren sobre su propio eje, al tratar de aplicar dicha información en el código, presenté muchos problemas que me tomaron un buen tiempo resolver.

A pesar de esto, pienso que la actividad propuesta es muy útil para familiarizarnos por completo con el uso de transformaciones matriciales. El tener que colocar, girar y trasladar tantas pirámides sin duda ayudó a que todos estos conceptos me quedaran mucho más claros y a practicar bastante con el manejo de las transformaciones `rotate` y `translate` sobre todo. Adicionalmente, es una buena introducción al manejo de distintas cámaras y a cómo debemos hacer para tener un buen control sobre la cámara libre.

La explicación del profesor fue adecuada ya que nos proporcionó todas las bases para poder realizar la mayor parte de la práctica sin ningún problema, pero si considero que sería bastante oportuno agregar a la explicación de clase algún ejercicio en el que se debe rotar una figura en todos sus ejes al mismo tiempo. Esto podría ahorrar mucho tiempo de trabajo a los alumnos sin darles la solución al ejercicio ya que ellos aún tendrían que buscar cómo aplicar esto a una pirámide.